

ML_期末報告 /機器學習智慧系統

學生姓名:

許允誠

系級:

財金三

學號:

112408540

目錄

目錄.....	1
壹、 摘要與導讀.....	3
1.1 研究背景與目的.....	3
1.2 原系統問題陳述.....	3
貳、 文獻回顧.....	5
2.1 ERP 財務控制架構的演進	5
2.2 財務異常檢測中的 AI 與機器學習	5
2.3 雲端原生 ERP 擴充架構	5
2.3 雲端原生 ERP 擴充架構	6
參、 目標與架構.....	7
3.1 已識別的研究缺口	7
肆、 研究方法.....	10
4.1 研究設計.....	10
4.2 實驗環境.....	10
4.3 評估指標.....	11
4.4. 機器學習模型配置.....	12
伍、 實作細節.....	13
5-1 智慧過帳驗證引擎 (Intelligent Posting Validation Engine, IPVE)	13
5.2. 基於 AI 的異常檢測服務 (AI-Based Anomaly Detection Service)	14
5.3. 自主對帳引擎 (Autonomous Reconciliation Engine)	15
5.4 預測性營收流失分析.....	16
5.6 安全性與合規架構.....	17
陸、 結論與心得.....	19

6.1 結論：從被動防禦到自主治理的財務數位轉型.....	19
6.2 個人心得與反思：架構美學與實證局限的綜合反思.....	19
柒、 參考文獻.....	21

壹、摘要與導讀

1.1 研究背景與目的

企業資源規劃 (ERP) 系統是全球企業的資訊系統基礎設施，處理企業的營運基礎，在電信、數位商務、醫療保健與金融等各領域每日有數千億筆資料的流動與傳輸；但是在目前機器語言學習與 AI 橫行的時代中，就算是 SAP S/4HANA 這樣大型的 ERP 系統，這些基本財務模型仍是受限於法規日程且被動的。

而本篇介紹的論文，作者希望透過 SAP BTP 這樣的雲端平台利用 RAP 建立 AI 驅動的財務自動化模組同時需要無縫接和原先的 ERP 財務與交易對帳系統，將原先勞力密集型、異常檢測大多為是後的傳統財務系統能應現代 AI 與透過演算學習自我監控調整，能夠預測與自動糾錯，不被外部監控與法規因為日程延後與其他拖延造成財務風險的未即時處理。

1.2 原系統問題陳述

傳統的 ERP 財務營運系統表現出四個問題：

- **靜態的基於規則驗證 (Static Rule-Based Validation)：** 財務過帳完全依賴於預先定義的驗證邏輯，無法在風險發生時進行適應與立即偵錯。
- **依賴期後對帳 (Post-Period Reconciliation Dependency)：** 明細分類帳到總帳的對帳通常是批次驅動且手動進行的。
- **延遲的異常檢測 (Delayed Anomaly Detection)：** 欺詐檢測和異常識別通常在發生資金風險暴露後才進行。
- **零碎的雲端擴充功能 (Fragmented Cloud Extensions)：** AI 功能通常作為鬆散耦合的外部服務提供，而不是嵌入式的交易控制。

這些限制會產生連鎖反應，在合規性與高交易量數位交易環境系統中，加劇財務風險暴露、營運開銷並導致效率低下。

為此，作者研究致力於設計、建構並量化評估一個全新的架構範式——「自主財務營運架構」 (Autonomous Financial Operations Framework, AFOF)，該架構將 AI 增強型智慧直接整合到雲端原生 ERP 財務工作流程中。

AFOF 目標如下：

- 設計一個具備擴展性與模組化的金融科技自動化架構。
- 運用基於機器學習的方法，在交易處理過程中建立一個應用程式內（in-app）的反應式異常檢測系統。
- 實現自主對帳與預測性營收流失檢測。
- 在支援智慧控制的同時，確保 ERP 效能的完整性。
- 與傳統基於規則的系統相比，量化評估其改善程度。

貳、文獻回顧

2.1 ERP 財務控制架構的演進

作者指出現在 ERP 系統根據歷史演進蛻變，目前 SAP S/4HANA 這樣的系統配備了可配置的驗證、替代邏輯與授權層級，以防止交易崩潰。儘管這些機制很強大，但它們也是靜態的，並且依賴於系統實作期間設定的固定條件。

ERP 現代化的變革性發展可見於記憶體內處理 (in-memory processing) 以及即時分析。然而，極少有現代化舉措針對核心交易工作流內的自適應智慧進行目標設定，同時兼顧效能與報表的最佳化。

2.2 財務異常檢測中的 AI 與機器學習

機器學習 (ML) 方法已成功應用於金融服務的以下領域：信用欺詐檢測、信用風險評分與交易異常檢測。邏輯斯迴歸、決策樹和梯度提升 (gradient boosting) 等方法在結構化財務數據中展現了強大的預測能力。像分群 (clustering) 和自編碼器 (autoencoders) 等非監督式方法也被用於大規模交易環境中的離群值檢測。

雖然它們在多數 AI 驅動的財務風險系統中表現出色，但它們是透過外部監控平台運作，消耗提取出的數據集，而不是與交易引擎進行通訊。這種分離增加了延遲，使修正自動化變得複雜，並且通常在執行緩解行動之前需要人員介入 (human in the loop)。

此外，現有文獻集中在銀行與支付處理系統上，而非 ERP 內嵌的財務控制。這種將即時 ML 推論整合到 ERP 驗證工作流中的做法，在學術界與產業界仍處於起步階段。

2.3 雲端原生 ERP 擴充架構

ERP 系統已經從單體式 (monolithic) 轉變為由 SAP Business Technology Platform 等平台所實現的雲端原生擴充性模型。這些平台鼓勵微服務架構、模組間的事件驅動通訊、API 優先設計以及作為容器進行部署。

近期的架構風格特徵為基於 REST 服務、事件網格 (event meshes) 以及微服務編排的鬆散耦合擴充。像 SAP ABAP RESTful 應用程式編程模型 (RAP) 這類

近期的行為驅動編程範式，支援具有交易一致性與基於 Fiori 介面展示的服務導向 ERP 開發。

在因疫情而使我們皆已轉向的雲端原生擴充模型中，我們實際的實作通常不會將重點放在將任何控制邏輯，帶入能夠獨立於業務功能而存在的自給自足營運能力之自動化上。然而，目前探索將 AI 模型結構性地嵌入此類擴充層，以實作自適應財務治理結構的研究仍相當有限。

2.3 雲端原生 ERP 擴充架構

企業研究一直關注於加速財務結帳週期。已部署了機器人流程自動化 (RPA) 與工作流自動化工具，以將手動的日記帳分錄處理和帳戶對帳工作降至最低。根據研究，自動化可將營運開銷降至最低，並提高結帳的準確性。

另一方面，RPA 解決方案僅透過模擬人類的操作在使用者介面 (UI) 層級上運作，而非重新設計控制架構。它們仍然是被動的且依賴於規則。它們無法憑空產生預測結果，也不會隨時持續學習交易行為模式是如何運作的。

在如訂閱計費與營收管理等高交易量環境中，對帳不匹配通常歸咎於時間差、整合延遲或動態定價調整。然而，目前缺乏專注於預測性對帳模型的研究，這些模型有助於在財務報表受到影響之前檢測出潛在的不匹配。

參、目標與架構

3.1 已識別的研究缺口

■ ERP 交易流程中缺乏 AI 加速：

目前的異常檢測系統是在 ERP 驗證邏輯之外運作。

■ 缺乏統一的架構框架：

從未有人將 ERP 核心系統、雲端原生擴充平台、AI 推論引擎與事件驅動的對帳服務整合到一個具有凝聚力的財務自動化架構中。

■ 量化基準測試有限：

很少有研究提供基於規則的 ERP 控制與 AI 增強型內嵌驗證系統之間企業級的效能比較。

■ 被動而非自主的財務控制：

目前的系統主要是在交易後檢測異常狀況，而不是在執行過程中預防或自主解決它們。

3.2 高階 AFOF 分層架構 (High Level AFOF Layered Architecture)

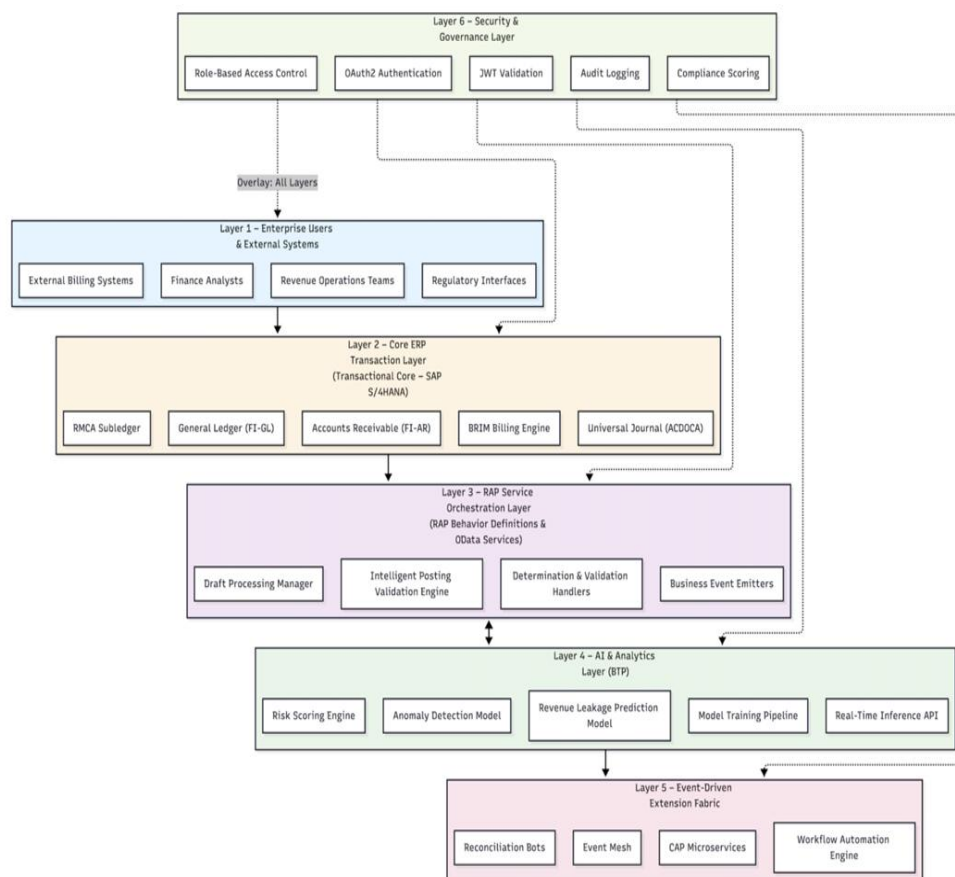


Fig 1: AFOF Layered Architecture

(圖表內容翻譯)

- **第 6 層 – 安全與治理層 (Layer 6 – Security & Governance Layer)** [覆蓋所有層級]
 - 基於角色的存取控制 (Role-Based Access Control)
 - OAuth2 身份驗證 (OAuth2 Authentication)
 - JWT 驗證 (JWT Validation)
 - 稽核日誌記錄 (Audit Logging)
 - 合規性評分 (Compliance Scoring)
- **第 1 層 – 企業使用者與外部系統 (Layer 1 – Enterprise Users & External Systems)**
 - 外部計費系統 (External Billing Systems)
 - 財務分析師 (Finance Analysts)
 - 營收營運團隊 (Revenue Operations Teams)
 - 監管介面 (Regulatory Interfaces)
- **第 2 層 – 核心 ERP 交易層 (Layer 2 – Core ERP Transaction Layer)**
(交易核心 – SAP S/4HANA)
 - RMCA 明細分類帳 (RMCA Subledger)
 - 總帳 (FI-GL) (General Ledger (FI-GL))
 - 應收帳款 (FI-AR) (Accounts Receivable (FI-AR))
 - BRIM 計費引擎 (BRIM Billing Engine)
 - 通用日記帳 (ACDOCA) (Universal Journal (ACDOCA))
- **第 3 層 – RAP 服務編排層 (Layer 3 – RAP Service Orchestration Layer)** (RAP 行為定義與 OData 服務)
 - 草稿處理管理員 (Draft Processing Manager)

- 智慧過帳驗證引擎 (Intelligent Posting Validation Engine)
- 決定與驗證處理常式 (Determination & Validation Handlers)
- 業務事件發送器 (Business Event Emitters)
- **第 4 層 – AI 與分析層 (Layer 4 – AI & Analytics Layer) (BTP)**
 - 風險評分引擎 (Risk Scoring Engine)
 - 異常檢測模型 (Anomaly Detection Model)
 - 營收流失預測模型 (Revenue Leakage Prediction Model)
 - 模型訓練管線 (Model Training Pipeline)
 - 即時推論 API (Real-Time Inference API)
- **第 5 層 – 事件驅動擴充架構 (Layer 5 – Event-Driven Extension Fabric)**
 - 對帳機器人 (Reconciliation Bots)
 - 事件網格 (Event Mesh)
 - CAP 微服務 (CAP Microservices)
 - 工作流自動化引擎 (Workflow Automation Engine)

肆、研究方法

4.1 研究設計

將執行實驗性比較評估方法，以研究提出的系統與目前 ERP 財務控制機制（基於規則）的差異。基準測試將比較在相同交易資料集上運行的兩種系統配置：

- **基準系統 (Baseline System)：** 配置標準 ERP 財務驗證與對帳邏輯的 SAP S/4HANA，使用基於規則的控制、替代規則以及期後對帳流程。
- **實驗系統 (AFOF)：** 一個具備 AI 增強功能的財務自動化架構，能夠透過智慧驗證服務、異常檢測模型以及自主對帳機制來減少時間消耗。這些機制由 SAP Business Technology Platform 提供動力，並使用 TVM - SAP ABAP RESTful 應用程式編程模型 (RAP) 進行實作

為確保可比性，兩種系統皆在同等的模擬企業交易場景下進行評估。

4.2 實驗環境

使用模擬的企業 ERP 環境，針對訂閱制與數位服務組織特有的高交易量財務交易場景進行模擬。該環境包含以下部分：

- ERP 核心財務模組（總帳、應收帳款、明細分類帳管理）
- 訂閱計費交易流
- 事件驅動型財務過帳模擬
- 即時交易驗證服務
- 雲端 AI 推論服務

表 2 資料集特徵

參數 (Parameter)	數值/內容 (Value)
每個模擬週期的總交易量	1,000 萬至 5,000 萬筆
交易類型	計費、調整、退款、營收認列
生成的分類帳分錄	約交易量的 1.8 倍
異常注入率	1-3%
營收流失情境	模擬 0.5 – 2%

4.3 評估指標

使用五個相關的效能指標來評估 AFOF 架構的有效性：

■ 財務結帳週期時間 (Financial Close Cycle Duration)

$$\text{CloseCycleReduction} = \frac{\text{BaselineCloseTime} - \text{AFOFCloseTime}}{\text{BaselineCloseTime}}$$

財務對帳與結帳活動所消耗的時間是以小時來衡量（這加重了完成結帳的工作負載）。此比率代表了自動化財務控制的營運效率。

■ 異常檢測準確性 (Anomaly Detection Accuracy)

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

AI 模型的異常檢測效能使用標準分類指標進行評估：

- 精準率 (Precision)
- 召回率 (Recall)
- F1 分數 (F1-score)
- TP = 檢測到的真陽性異常 (True Positive anomalies detected)
- FP = 偽陽性 (False positives)
- FN = 未檢測到的異常 (Undetected anomalies)

■ 手動對帳工作量 (Manual Reconciliation Effort)

$$\text{EffortReduction} = \frac{\text{BaselineHours} - \text{AFOFHours}}{\text{BaselineHours}}$$

手動對帳工作量被評估為分析師解決明細分類帳與總帳過帳之間差異所需的小時數。

■ 營收流失風險暴露 (Revenue Leakage Exposure)

$$LeakageRate = \frac{UncorrectedRevenueErrors}{TotalRevenueTransactions}$$

它被定義為在財務報表產出前，未能被自動更正的計費差異百分比。

■ 交易處理延遲 (Transaction Processing Latency)

$$LatencyImpact = AFOFTransactionTime - BaselineTransactionTime$$

為了確保 AI 的整合不會降低 ERP 的效能，我們測量了平均交易處理延遲。延遲測量結果以毫秒 (millisecond) 為單位進行記錄。

4.4. 機器學習模型配置

從交易記錄中提取的特徵包括：

- 過帳金額變異 (Posting amount variance)
- 帳戶頻率模式 (Account frequency patterns)
- 計費週期時間 (Billing cycle timing)
- 客戶交易歷史 (Customer transaction history)
- 分類帳對應一致性 (Ledger mapping consistency)
- 事件排序模式 (Event sequencing patterns)

伍、實作細節

自主財務營運架構 (AFOF)，這是一個模組化、雲端原生的架構，將企業 ERP 交易服務、基於 AI 的分析解決方案，以及事件驅動的自動化解決方案連結起來。此實作利用 SAP S/4HANA 作為交易核心，並使用其擴充功能；同時將基於 SAP Business Technology Platform 的進階分析與編排組件進行部署。

系統架構的概貌如下所示，其包含五個主要組件：

- 智慧過帳驗證引擎 (Intelligent Posting Validation Engine, IPVE)
- 基於 AI 的異常檢測服務 (AI-Based Anomaly Detection Service)
- 自主對帳引擎 (Autonomous Reconciliation Engine)
- 預測性營收流失分析模組 (Predictive Revenue Leakage Analytics Module)
- 事件驅動擴充層 (Event-Driven Extension Layer)

這些組件中的每一個皆獨立運行，同時透過基於事件的訊息系統與 REST API 進行通訊。

5-1 智慧過帳驗證引擎 (Intelligent Posting Validation Engine, IPVE)

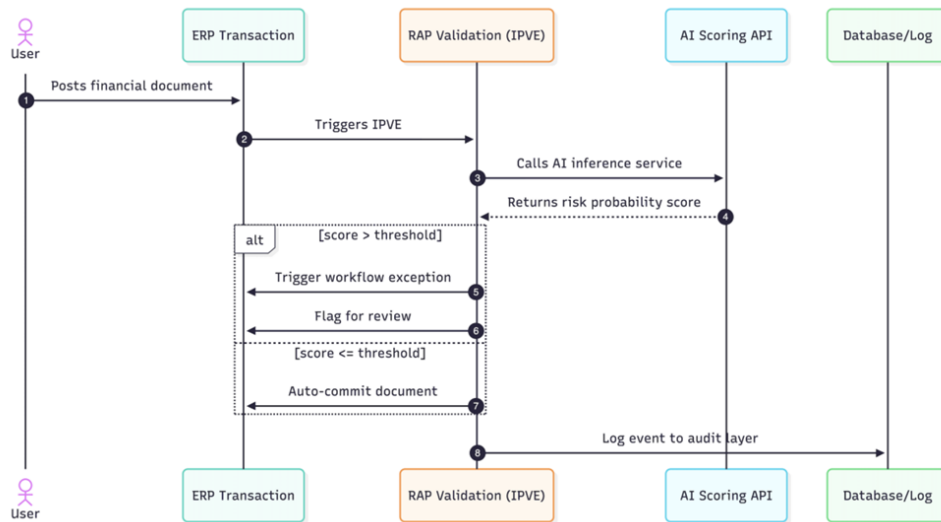


圖 3：智慧過帳驗證序列圖

IPVE 介入 (hooks into) 財務文件的創建過程，並在執行資料庫提交 (commit) 操作之前進行驗證。該引擎結合了機器學習的結果來確定交易風險，而不是僅使用傳統的基於規則的驗證。

5.2. 基於 AI 的異常檢測服務 (AI-Based Anomaly Detection Service)

異常檢測服務作為託管於雲端的推論引擎，它根據訓練好的機器學習模型來分析交易特徵。

資料管線 (Data Pipeline)

資料處理管線包含：

- 交易資料提取 (Transaction data extraction)
- 特徵轉換 (Feature transformation)
- 模型推論 (Model inference)
- 風險評分 (Risk scoring)
- 回應生成 (Response generation)

模型分析的特徵例如：

- 交易金額偏差 (Transaction amount deviation)
- 歷史帳戶行為 (Historical account behavior)
- 計費頻率模式 (Billing frequency patterns)
- 事件序列異常 (Event sequence anomalies)
- 分類帳對應不一致 (Ledger mapping inconsistencies)

目前，推論服務公開了 RESTful API，這些 API 接收交易詮釋資料 (metadata)，並輸出一個介於 (0, 1) 之間的機率性異常分數。

回應結構範例 JSON (Example response structure)

```
{  
  
  "transaction_id": "TX784392",  
  
  "risk_score": 0.82,  
  
  "classification": "High Risk"  
}
```

5.3. 自主對帳引擎 (Autonomous Reconciliation Engine)

利用機器人流程自動化 (RPA) 技術，在一個追蹤分類帳同步事件的事件驅動對帳服務上，將財務對帳流程自動化。

對帳邏輯 (Reconciliation Logic)

對帳引擎持續評估以下各項之間的一致性：

- 明細分類帳交易 (Subledger transactions)
- 總帳分錄 (General ledger entries)
- 計費系統輸出 (Billing system outputs)
- 營收認列過帳 (Revenue recognition postings)

事件處理 workflow (Event Processing Workflow)

1. 從 ERP 系統發出一個交易過帳事件。
2. 事件訊息層將該事件轉發至對帳服務。
3. 分類帳比較演算法評估差異。
4. 當檢測到不匹配時，系統會執行以下其中一項操作：
 - 自動調整過帳 (Automatic adjustment posting)
 - 創建對帳分錄 (Reconciliation entry creation)
 - workflow 升級以供人工審查 (Workflow escalation for human review)

對帳引擎會針對每一次的自動更正維護稽核日誌。

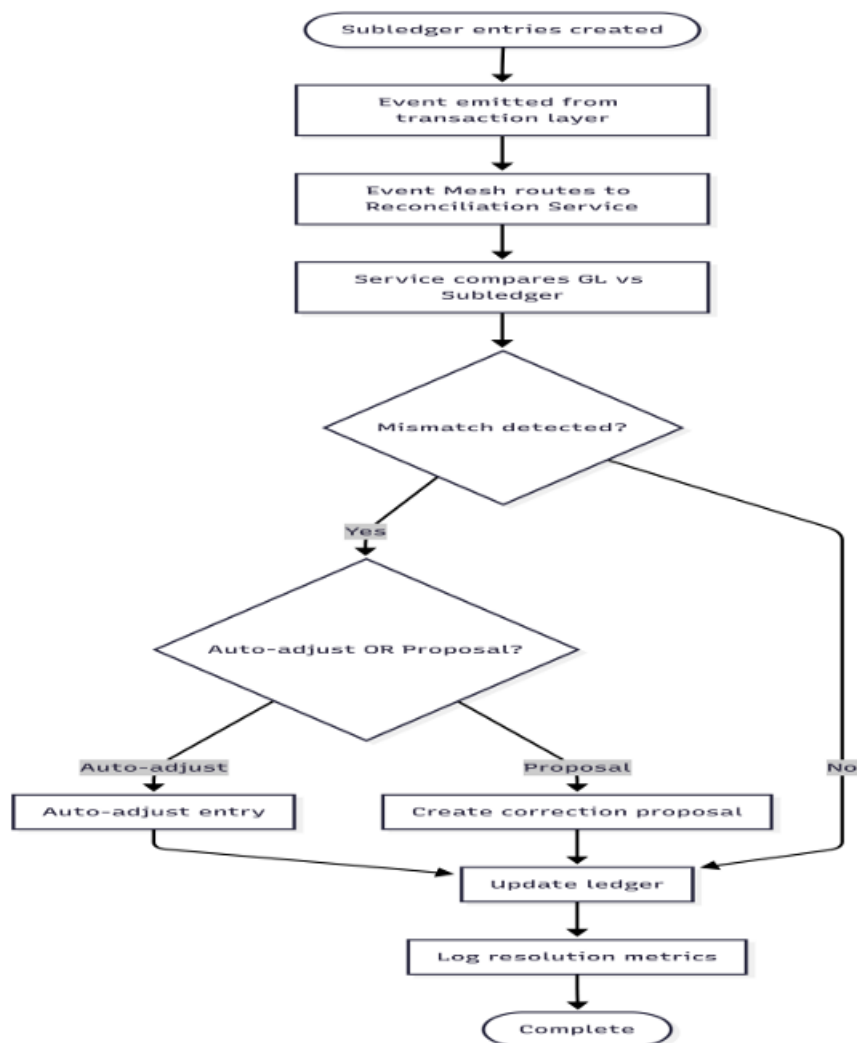


圖 4 自主對帳流程

5.4 預測性營收流失分析

實作了用於檢測營收流失的預測分析模組，以在財務報告週期到來之前，識別出計費資料中潛在的不匹配狀況。

特徵輸入 (Feature Inputs)

模型會在計費與營收認列資料中測試各種模式，例如：

- 訂閱生命週期事件 (Subscription lifecycle events)
- 定價模型變異 (Pricing model variations)
- 計費頻率偏差 (Billing frequency deviations)
- 折扣規則不一致 (Discount rule inconsistencies)

- 使用量與計費的關聯性 (Usage-to-billing correlation)

預測性評分

分析引擎會將流失機率分數分配給交易叢集。超過指定閾值的交易會觸發警報或自動化修復流程。

此方法使系統能夠主動解決營收問題，而不是在生成財務報表後才做出回應。

5.5 事件驅動擴充層

該架構旨在透過其事件驅動擴充層來支援即時自動化與解耦架構，該擴充層掌管著 ERP 服務與不同地端 (on-premise) 及雲端組件之間的互動。

事件通訊模型 (Event Communication Model)

系統採用非同步訊息傳遞模式，其中 ERP 交易事件會被發布到一個事件分發服務。

事件類型包含：

- 財務文件已創建 (Financial document created)
- 計費事件已處理 (Billing event processed)
- 檢測到對帳不匹配 (Reconciliation mismatch detected)
- 異常分類已完成 (Anomaly classification completed)

訂閱的微服務會處理這些事件，並執行特定任務，例如分析執行或對帳邏輯。

5.6 安全性與合規架構

框架的所有層級皆實作了安全機制，以確保符合企業財務治理標準。

關鍵機制包含：

- 針對 API 服務基於 OAuth 的身分驗證 (OAuth-based authentication for API services)
- 基於角色的授權控制 (Role-based authorization controls)
- ERP 與雲端服務之間的加密通訊 (Encrypted communication between ERP and cloud services)

- 交易級別的稽核日誌記錄 (Transaction-level audit logging)
- 自動化合規性評分 (Automated compliance scoring)

這些機制確保智慧自動化不會損害監管要求或財務可稽核性。

5.7 實作結果

此研究的實驗結果顯示，所提出的架構顯著提高了營運效率，同時也提升了對帳自動化與異常檢測的準確性，同時在 SAP S/4HANA ERP 等企業環境中保留了可接受的交易處理延遲。

實驗評估顯示，所提出的架構為企業財務系統提供了可量化的營運效益：

- 財務結帳週期時間減少 43%
- 異常檢測準確度提高 37%
- 手動對帳工作量減少 52%
- 營收流失風險暴露減少 28%
- 極小的交易延遲增加 (8 毫秒)

這些結果證實了使用 SAP Business Technology Platform 等現代雲端原生擴充架構，將智慧財務自動化直接嵌入 ERP 交易工作流中的能力與效能優勢。

陸、結論與心得

6.1 結論：從被動防禦到自主治理的財務數位轉型

本研究探討 Raghavendra Depa 所提出的「自主財務營運架構 (AFOF)」，成功展示了現代企業財務從傳統的「事後被動救火」向「事前主動防禦」的典範轉移。AFOF 透過在 Layer 3 (RAP 服務編排層) 實作智慧過帳驗證引擎 (IPVE)，實時攔截高產量的交易憑證，並在不破壞 ERP 核心穩定的前提下 (平均僅增加 8 毫秒的極低延遲)，完成 AI 異常風險評估。

此外，該架構利用 Layer 5 (事件驅動擴充層) 的 Event Mesh 技術，將沉重的跨系統多方數據核對工作移至背景非同步執行。實證數據顯示，該模型成功降低了 52% 的人工對帳工作量，並提升了 37% 的異常檢測精準度，為現代雲端原生 ERP 財務模組的高效、合規營運樹立了全新標竿。

6.2 個人心得與反思：架構美學與實證局限的綜合反思

貫穿本學期從期中到期末的兩篇 SAP 技術論文研究，為我拼湊出了現代智慧企業 (Intelligent Enterprise) 的完整視野。期中報告探討了 Siar Sarferaz 所提出的頂層生成式 AI 治理戰略，思考如何為企業導入 AI 制定安全指引；而期末報告則聚焦於 Raghavendra Depa 的 AFOF 架構，向我展示了底層技術硬核落地的可能性。從這兩者的對比與結合中，我獲得了以下三點深刻的反思：

1. 「空間換取時間」的架構美學 身為管院的學生，我過去常疑惑：如何在引入機器學習繁重算力的同時，不拖垮 ERP 這類要求極致並發與絕對穩定的核心系統？AFOF 巧妙地利用 SAP BTP 與 RAP 的外掛式優化 (Side-by-Side Extension) 給予了解答。它讓過帳前的同步攔截保持極輕量，而將複雜的預測性營收流失分析與對帳 Bots 拋到雲端非同步執行。這種「用雲端空間換取核心時間」的解耦設計，完美平衡了效能與智慧，是非常高明的工程實踐。
2. 批判性審視：工業界論文的「黑箱實證」限制 然而，秉持學術研究的嚴謹態度，本篇論文亦存在一個不容忽視的局限性，即實證數據與演算法開源的「不可得性」。作者雖然宣稱在 1,000 萬至 5,000 萬筆高產量交易的模擬環境中取得了極佳成果，但基於企業資料隱私與 SAP 商業機密，論文並未提供原始數據集 (Raw Datasets) 或測試代碼。

這種缺乏公開 Benchmark 進行交叉檢證的「黑箱實證」，使得外部研究者難以對其結論進行複製（Reproducibility）與壓力測試。這正好呼應了期中報告裡 Sarferaz 博士所強調的「AI 治理與模型審計」：如果一個智慧財務系統的量化評估無法被客觀重複檢驗，那麼企業在引進此架構時，就必須承擔模型在不同產業環境下可能產生偏見或失效的潛在金融風險。

未來財金人員的思維轉型 當 AFOF 自動化了絕大多數的常規對帳，甚至能提前抓出複雜訂閱制下的營收流失時，傳統會計與財務人員「靠肉眼查帳」的價值將徹底被時代淘汰。正如 AFOF 流程圖中所揭示的「智慧異常處理層（Human-in-the-Loop）」，未來的財金精英不再是資料的操作員，而是高階的治理者。我們必須學會如何去審查 AI 所提出的複雜合約修正建議，並具備監控機器學習模型精準率（Precision）與召回率（Recall）的跨領域能力。

柒、 參考文獻

- [1] R. Depa, "Design and Implementation of an Autonomous Financial Operations Framework (AFOF)," SAP Application Engineering.
- [2] S. Sarferaz, "Implementing Generative AI Into ERP Software," SAP SE, Research and Development Department, 69190 Walldorf, Germany.
- [3] B. Kitchenham and S. M. Chartes, "Guidelines for performing systematic literature reviews in software engineering," EBSE, Softw. Eng. Group School Comput. Sci. Math., Keele Univ., Keele, U.K., Tech. Rep. EBSE2007-01, 2007.
- [4] Q. Lu, L. Zhu, X. Xu, Y. Liu, Z. Xing, and J. Whittle, "A taxonomy of foundation model based systems through the lens of software architecture," in Proc. IEEE/ACM 3rd Int. Conf. AI Eng.-Softw. Eng. AI, Apr. 2024, pp. 1–6.
- [5] H. Kazem and D. Safaei, "Generative artificial intelligence for enterprise platforms," in Proc. 32nd Eur. Conf. Inf. Syst. (ECIS), Jun. 2024, pp. 13–19. [6] H. Mahmoud, H. M. Elbadawy, T. Ismail, and D. Mi, "A comprehensive review of generative AI applications in 6G," in Proc. 6th Novel Intell. Lead. Emerg. Sci. Conf. (NILES), Oct. 2024, pp. 593–596.
- [7] A. Faccia, F. Manni, V. Pandey, and L. P. L. Cavaliere, "Blazing a new trail in ERP integration with NLP and generative AI through APIs: A fraud examination perspective," in Proc. 8th Int. Conf. Inf. Syst. Eng., Dec. 2023, pp. 177–184.
- [8] N. Rane, S. Choudhary, and J. Rane, "Intelligent manufacturing through generative artificial intelligence, such as ChatGPT or bard," Univ. Mumbai, Maharashtra, India, Tech. Rep. 4681747, Jan. 2024.
- [9] P. Arora, L. Desu, A. Kumar, R. Kumar, and A. Marinescu, "Enhancing profitability through AI-optimized accounts receivable: Reducing cash conversion cycles," in Proc. Int. Conf. Electr., Comput. Energy Technol. (ICECET), Jul. 2024, pp. 1–5.
- [10] H. Aguinis, J. R. Beltran, and A. Cope, "How to use generative AI as a human resource management assistant," *Organizational Dyn.*, vol. 53, no. 1, Jan. 2024, Art. no. 101029.

[11] G. Anshumaan and D. Sharma, "Generative AI for software test modelling with a focus on ERP software," in Proc. Int. Conf. Adv. Comput., Commun. Inf. Technol. (ICAICCIT), Nov. 2023, pp. 187–193.